

[House] House Price Predictor -- Linear Regression from Scratch

***Session Goal:** Understand *what* Linear Regression is, *why* it works, and *how* we use it to predict house prices -- then deploy the trained model to Hugging Face.*

[List] What We'll Cover

Step	Topic
1	What is Machine Learning?
2	What is Linear Regression? (the math, simply explained)
3	Generate our housing dataset
4	Explore & visualise the data
5	Train the model
6	Understand what the model learned (coefficients)
7	Evaluate: MAE, RMSE, R^2
8	Make predictions on new houses
9	Push to Hugging Face Hub

Step 0 -- Install Dependencies

Run this first. Colab already has most of these, but we pin them to be safe.

```
!pip install pandas numpy scikit-learn matplotlib seaborn huggingface_hub --quiet
print("[OK] All packages installed.")
```

Step 1 -- What is Machine Learning?

Traditional programming:
```
Rules + Data  Output
```

You write the formula yourself.

Machine Learning:
```
Data + Output  Rules (learned automatically)
```

```

The algorithm *finds* the formula for you from examples.

In our case:
- **Data** = 300 houses with features (size, bedrooms, bathrooms, age, location)
- **Output** = price in  Lakhs
- **Goal** = learn the formula so we can predict price for *new* houses

---

## Step 2 -- What is Linear Regression?

The simplest ML algorithm. The core idea: **draw the best straight line through the data.**

### Simple version (1 feature):

y_pred = w1 x1 + b

- $x1$ = size of house
- $y_pred$ = predicted price
- $w1$ = **weight** (how much each sq ft adds to price)
- $b$ = **bias/intercept** (base price even if size = 0)

### Our version (5 features):

y_pred = w1(size) + w2(bedrooms) + w3(bathrooms) + w4(age) + w5(location) + b

### How does it *learn* the weights?

It tries to **minimise the error** between predicted price $y_pred$ and actual price $y$.

Error measure = **Mean Squared Error (MSE)**:
MSE = 1/n sum_{i=1}^{n} (y_i - y_pred_i)^2

Think of it as: *"How wrong am I on average? Now adjust weights to be less wrong."*

---

## Step 3 -- Generate the Dataset

We create **synthetic (fake but realistic) data** using the true formula we know:

```

price = 15 + (size 0.05) + (bedrooms 5) + (bathrooms 3) + (location 4) - (age 0.5) + noise
```

The model's job is to *rediscover* approximately these coefficients just from the data.

```
import numpy as np
import pandas as pd
```
```

```
import os

np.random.seed(42)
n = 300

# Generate features
size_sqft      = np.random.uniform(500, 4000, n)
bedrooms       = np.random.randint(1, 7, n)
bathrooms      = np.random.randint(1, 5, n)
age_years      = np.random.randint(0, 41, n)
location_score = np.random.randint(1, 11, n)

# TRUE formula (this is what we're trying to learn)
price_lakhs = (
    15.0
    + (size_sqft      * 0.05)
    + (bedrooms       * 5.0)
    + (bathrooms      * 3.0)
    + (location_score * 4.0)
    - (age_years      * 0.5)
)

# Add realistic noise (market randomness)
noise = np.random.normal(0, 12, n)
price_lakhs = np.clip(price_lakhs + noise, 10.0, None)

df = pd.DataFrame({
    'size_sqft':      np.round(size_sqft, 2),
    'bedrooms':       bedrooms,
    'bathrooms':      bathrooms,
    'age_years':      age_years,
    'location_score': location_score,
    'price_lakhs':    np.round(price_lakhs, 2)
})

os.makedirs('data', exist_ok=True)
df.to_csv('data/house_prices.csv', index=False)

print(f"[OK] Dataset: {len(df)} rows   {len(df.columns)} columns")
print(f"   Price range: Rs. {df.price_lakhs.min():.1f}L -- Rs. {df.price_lakhs.max():.1f}L")
df.head()
```

Step 4 -- Explore & Visualise the Data

Before training, always **look at your data**. We want to understand:

- The distribution of prices

- Which feature correlates most with price

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style='whitegrid', palette='muted')
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# 1. Price distribution
axes[0].hist(df['price_lakhs'], bins=25, color='#4F46E5', alpha=0.8, edgecolor='white')
axes[0].set_title('Price Distribution ( Lakhs)', fontweight='bold')
axes[0].set_xlabel('Price (Rs.   Lakhs)')
axes[0].set_ylabel('Count')
```

```
# 2. Size vs Price scatter
axes[1].scatter(df['size_sqft'], df['price_lakhs'], alpha=0.4, color='#4F46E5', s=20)
axes[1].set_title('Size vs Price', fontweight='bold')
axes[1].set_xlabel('Size (sq ft)')
axes[1].set_ylabel('Price (Rs.   Lakhs)')

# 3. Location Score vs Price
axes[2].scatter(df['location_score'], df['price_lakhs'], alpha=0.4, color='#059669', s=20)
axes[2].set_title('Location Score vs Price', fontweight='bold')
axes[2].set_xlabel('Location Score (1-10)')
axes[2].set_ylabel('Price (Rs.   Lakhs)')

plt.tight_layout()
plt.show()

print("\n[Data] Summary Statistics:")
print(df.describe().round(2))
```

```
# Correlation heatmap -- which features are most related to price?
plt.figure(figsize=(8, 5))
corr = df.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            square=True, linewidths=0.5, cbar_kws={'shrink': 0.8})
plt.title('Feature Correlation Matrix', fontweight='bold', pad=12)
plt.tight_layout()
plt.show()

print("\n[Key] Correlation with price:")
print(corr['price_lakhs'].sort_values(ascending=False).drop('price_lakhs').to_string())
```

Step 5 -- Train the Linear Regression Model

Key concept: Train/Test Split

We hold back 20% of data the model never sees during training.

This tests whether it generalises to *new* houses, not just ones it memorised.

```
300 houses
|-- 240 Training set    model learns from these
|__  60 Test set        we check predictions on these
```

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib, json

features = ['size_sqft', 'bedrooms', 'bathrooms', 'age_years', 'location_score']
target   = 'price_lakhs'

X = df[features]
y = df[target]

# 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
print(f"Training rows: {len(X_train)} | Test rows: {len(X_test)}")

# === TRAIN === (this is the ONE line that does all the learning)
model = LinearRegression()
model.fit(X_train, y_train)

print("\n[OK] Model trained!")
print("   Under the hood, sklearn solved the Ordinary Least Squares equation:")
print("   w = (X'X)^-1 X'y")
```

Step 6 -- What Did the Model Learn?

After training, the model has learned **weights (coefficients)** for each feature.

Let's compare them to our **true** coefficients we used to generate the data.

```
# True coefficients we used
true_coefs = {
    'size_sqft': 0.05, 'bedrooms': 5.0, 'bathrooms': 3.0,
    'age_years': -0.5, 'location_score': 4.0
}

coef_df = pd.DataFrame({
    'Feature':       features,
    'True Coef':     [true_coefs[f] for f in features],
    'Learned Coef':  [round(c, 4) for c in model.coef_]
})
print("[Math] Intercept (b)  -- True: 15.00  |  Learned:", round(model.intercept_, 2))
print()
print(coef_df.to_string(index=False))

# Visualise
x_pos = range(len(features))
fig, ax = plt.subplots(figsize=(10, 5))
bar_w = 0.35
ax.bar([p - bar_w/2 for p in x_pos], [true_coefs[f] for f in features],
       width=bar_w, label='True', color='#6EE7B7', edgecolor='white')
ax.bar([p + bar_w/2 for p in x_pos], model.coef_,
       width=bar_w, label='Learned', color='#4F46E5', edgecolor='white', alpha=0.85)
ax.set_xticks(list(x_pos))
ax.set_xticklabels(features)
ax.axhline(0, color='gray', linewidth=0.8)
ax.set_title('True vs Learned Coefficients', fontweight='bold')
ax.set_ylabel('Coefficient Value')
ax.legend()
plt.tight_layout()
plt.show()

print("\n[Insight] Key insight: The model learned similar coefficients to the TRUE formula")
print("   just by looking at the data -- no formula was given!")
```

Step 7 -- Evaluate the Model

Three standard metrics:

| Metric | Formula | Meaning |

|--------|---------|----------|

| **MAE** | avg \|actual predicted\| | Average rupee error (Lakhs) |

| **RMSE** | (avg (actual predicted)^2) | Penalises big errors more |

| **R^2** | 1 (SS_res / SS_tot) | % of price variation explained (1.0 = perfect) |

```
y_pred = model.predict(X_test)

mae  = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2   = r2_score(y_test, y_pred)

print("=" * 40)
print("  MODEL EVALUATION RESULTS")
print("=" * 40)
print(f"  MAE  : Rs. {mae:.2f}  Lakhs")
print(f"  RMSE : Rs. {rmse:.2f}  Lakhs")
print(f"  R^2   : {r2:.4f}  ({r2*100:.1f}% of variance explained)")
print("=" * 40)

# Visual: Actual vs Predicted
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Scatter: Actual vs Predicted
axes[0].scatter(y_test, y_pred, alpha=0.5, color='#4F46E5', s=30)
lims = [min(y_test.min(), y_pred.min()) - 5, max(y_test.max(), y_pred.max()) + 5]
axes[0].plot(lims, lims, 'r--', linewidth=1.5, label='Perfect prediction')
axes[0].set_xlabel('Actual Price (Rs.   Lakhs)')
axes[0].set_ylabel('Predicted Price (Rs.   Lakhs)')
axes[0].set_title('Actual vs Predicted', fontweight='bold')
axes[0].legend()

# Residuals
residuals = y_test.values - y_pred
axes[1].hist(residuals, bins=20, color='#4F46E5', alpha=0.75, edgecolor='white')
axes[1].axvline(0, color='red', linestyle='--', linewidth=1.5)
axes[1].set_xlabel('Residual (Actual   Predicted)')
axes[1].set_ylabel('Count')
axes[1].set_title('Residual Distribution (should be centred at 0)', fontweight='bold')

plt.tight_layout()
plt.show()
```

Step 8 -- Predict on New Houses

Now let's use the model the same way the **web app** does -- give it custom inputs and get a price.

```
# Define a few test houses
test_houses = pd.DataFrame([
    {'size_sqft': 1850, 'bedrooms': 3, 'bathrooms': 2,   'age_years': 8,  'location_score': 8.2,
'description': 'Starter home'},
    {'size_sqft': 3500, 'bedrooms': 5, 'bathrooms': 4,   'age_years': 2,  'location_score': 9.5,
'description': 'Luxury villa'},
    {'size_sqft': 700,  'bedrooms': 2, 'bathrooms': 1,   'age_years': 30, 'location_score': 4.0,
'description': 'Old apartment'},
    {'size_sqft': 2200, 'bedrooms': 4, 'bathrooms': 3,   'age_years': 5,  'location_score': 7.0,
'description': 'Family home'},
])

X_new = test_houses[features]
```

```
predictions = model.predict(X_new)

print("[House] Predictions for New Houses:\n")
for i, (_, row) in enumerate(test_houses.iterrows()):
    print(f"  [{row['description']}]")
         print(f"         Size:  {int(row['size_sqft'])}  sqft  |  {int(row['bedrooms'])}  bed  |
{row['bathrooms']} bath | "
          f"Age: {int(row['age_years'])}yr | Location: {row['location_score']}")
    print(f"    -> Predicted Price: Rs. {predictions[i]:.2f}  Lakhs")
    print()
```

Step 9 -- Save Artifacts & Push to Hugging Face

We save:

- `model.pkl` -- the trained model object

- `feature_names.json` -- the ordered list of features the API needs

Then push both to `ritesh1918/house-price-predictor` on Hugging Face Hub.

```
os.makedirs('model', exist_ok=True)

# Save model
joblib.dump(model, 'model/model.pkl')

# Save feature list (ORDER MATTERS -- backend uses this)
with open('model/feature_names.json', 'w') as f:
    json.dump(features, f)

print("[OK] model/model.pkl saved")
print("[OK] model/feature_names.json saved")
print(f"   Features: {features}")
```

```
# Login to Hugging Face
# Run this cell -- a widget will appear asking for your HF token
from huggingface_hub import notebook_login
notebook_login()
```

```
from huggingface_hub import HfApi

api = HfApi()
username = api.whoami()['name']
repo_id  = f"{username}/house-price-predictor"

print(f"  Pushing to: https://huggingface.co/{repo_id}")

api.create_repo(repo_id=repo_id, exist_ok=True, repo_type="model")

api.upload_folder(
    folder_path="model",
    repo_id=repo_id,
    repo_type="model",
    commit_message="Upload trained Linear Regression model from Colab"
)

print(f"\n[OK] Model pushed! View it at:")
print(f"   https://huggingface.co/{repo_id}")
```

[Summary] Summary -- What We Learned

Concept	What it means
Linear Regression	Finds the best-fit line: `price = w1 size + w2 beds + ... + b`
Weights (w)	How much each feature contributes to the price
Intercept (b)	Base price when all features are zero
Train/Test Split	80% to learn, 20% to verify it generalises
MAE	Average prediction error in Lakhs
R^2	How much of the price variation our model explains
Hugging Face	Where we store the trained model so the web app can download it

[Launch] The Full Pipeline

```
Generate Data  Explore  Train  Evaluate  Save  Deploy to HF  Web API  UI
```

The web app at http://localhost:8000/ lets anyone predict prices using the model you just trained!